

\$\$\$ src.controle.projeto

```
from util.persistência_arquivo import carregar_arquivo, salvar_arquivo
from entidades.seguradora import get_seguradoras, set_seguradoras
from entidades.sinistro import get_sinistros, set_sinistros
from entidades.orcamento import get_orçamentos, set_orcamentos
from interfaces.interface_textual import loop_opções_execução
```

```
nome_arquivo = 'conjuntos_globais'
```

```
def salvar_aplicação():
    orçamento_peças = []
    orçamento_peças.append(get_seguradoras())
    orçamento_peças.append(get_sinistros())
    orçamento_peças.append(get_orçamentos())
    salvar_arquivo(nome_arquivo, objetos=orçamento_peças)
```

```
def recuperar_aplicação():
    orçamento_peças = carregar_arquivo(nome_arquivo)
    if orçamento_peças is not None:
        set_seguradoras(orçamento_peças[0])
        set_sinistros(orçamento_peças[1])
        set_orcamentos(orçamento_peças[2])
```

```
if __name__ == '__main__':
    recuperar_aplicação()
    loop_opções_execução()
    salvar_aplicação()
```

\$\$\$ src.entidades.seguradora

```
seguradoras = {}
```

```
def get_seguradoras():
```

```
    return seguradoras
```

```
def inserir_seguradora(seguradora):
```

```
    nome_seguradora = seguradora.nome
```

```
    if nome_seguradora not in seguradoras.keys():
```

```
        seguradoras[nome_seguradora] = seguradora
```

```
    return True
```

```
    else:
```

```
        print('Seguradora ' + nome_seguradora + ' tem cadastro')
```

```
    return False
```

```
def set_seguradoras(seguradoras1):
```

```
    global seguradoras
```

```
    seguradoras = seguradoras1
```

```
class Seguradora:
```

```
    def __init__(self, nome, cidade, cobertura_percentual):
```

```
        self.nome = nome
```

```
        self.cidade = cidade
```

```
        self.cobertura_percentual = cobertura_percentual
```

```
    def __str__(self):
```

```
        formato = '{} {:.<18} {} {:.<14} {} {:.<3} {}'
```

```
        seguradora_formatada = formato.format('|', self.nome, '|', self.cidade, '|',  
str(self.cobertura_percentual), '|')
```

```
        return seguradora_formatada
```

\$\$\$ src.entidades.peca

class Peca:

```
def __init__(self, código, nome, categoria, preco, mão_obra_própria):
    self.código = código
    self.nome = nome
    self.categoria = categoria if categoria in ('Original', 'Genuína', 'OEM') else 'indefinida'
    self.preco = preco
    self.mão_obra_própria = mão_obra_própria
```

```
def __str__(self):
    if self.mão_obra_própria: mão_obra_própria_str = ' Mão de obra Própria |'
    else: mão_obra_própria_str = ""
    formato = ' {} {:<15} {} {:<12} {} {:<6} {} {}'
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',
mão_obra_própria_str)
    return peca_formatado
```

class PeçaMecânica(Peca):

```
def __init__(self, código, nome, categoria, preco, tipo, dias_garantia, mão_obra_própria):
    super().__init__(código, nome, categoria, preco, mão_obra_própria)
    self.tipo = tipo if tipo in ('suspensão', 'direção', 'motor') else 'indefinida'
    self.dias_garantia = dias_garantia
```

```
def __str__(self):
    if self.mão_obra_própria: mão_obra_própria_str = ' Mecânico Próprio |'
    else: mão_obra_própria_str = ""
    formato = ' {} {:<15} {} {:<8} {} {:<12} {} {:<9} {} {:>8} {} {:>20}'
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',
self.tipo, '|', self.dias_garantia, '|', mão_obra_própria_str)
    return peca_formatado
```

class PeçaLataria(Peca):

```
def __init__(self, código, nome, categoria, preco, tipo, cor, mão_obra_própria):
    super().__init__(código, nome, categoria, preco, mão_obra_própria)
    self.tipo = tipo if tipo in ('externo', 'interno') else 'indefinida'
```

```
self.cor = cor
```

```
def __str__(self):
```

```
    if self.mão_obra_própria: mão_obra_própria_str = ' Funileiro Próprio |'
```

```
    else: mão_obra_própria_str = ''
```

```
    formato = ' {} {:<15} {} {:<8} {} {:<12} {} {:<9} {} {:>8} {} {:>20}'
```

```
    peca_formatado = formato.format('|', self.nome, '|', self.categoria, '|', f'R$ {self.preco:.2f}', '|',  
self.tipo, '|', self.cor, '|', mão_obra_própria_str)
```

```
    return peca_formatado
```

\$\$\$ src.entidades.orcamento

```
from entidades.seguradora import get_seguradoras
from entidades.sinistro import get_sinistros
from entidades.peca import PeçaMecânica, PeçaLataria

orcamentos = []

def get_orçamentos():
    return orcamentos

def inserir_orçamento(orçamento):
    if orçamento not in orcamentos:
        orcamentos.append(orçamento)
    else:
        print ('Orçamento de Pecas tem cadastro --- ' + str(orçamento))

def criar_orçamento(numero_sinistro, nome_seguradora, data):
    sinistro = get_sinistros().get(numero_sinistro)
    if sinistro is None:
        print('Sinistro ' + str(numero_sinistro) + ' não cadastrado')
        return

    seguradora = get_seguradoras().get(nome_seguradora)
    if seguradora is None:
        print('Seguradora ' + str(nome_seguradora) + ' não cadastrada')
        return

    orçamento = Orcamento(sinistro, seguradora, data)
    inserir_orçamento(orçamento)
    return orçamento

def set_orçamentos(orcamentos1):
    global orcamentos
    orcamentos = orcamentos1

def filtrar_orçamentos(data_mínima_orçamento, valor_máximo_peca,
```

```

cobertura_mínima_seguradora, prefixo_telefone_cliente, categoria,
    tipo_peça_mecânica, tipo_peça_lataria):
orçamentos_selecionados = []
for orçamento in orcamentos:
    if data_mínima_orcamento is not None and orçamento.data < data_mínima_orcamento:
        continue

    pecas_orçamento = list(orçamento.sinistro.pecas.values())

    if any(valor_máximo_peca is not None and peca.preco > valor_máximo_peca for peca in
pecas_orçamento):
        continue

    if (cobertura_mínima_seguradora is not None
        and orçamento.seguradora.cobertura_percentual < cobertura_mínima_seguradora):
        continue

    if prefixo_telefone_cliente is not None and not
orçamento.sinistro.telefone.startswith(str(prefixo_telefone_cliente)):
        continue

    if categoria is not None and any(peca.categoria != categoria for peca in pecas_orçamento):
        continue

    if tipo_peça_mecânica is not None:
        if any(
            not peca_mecânica_atende_filtros(
                peca,
                categoria,
                tipo_peça_mecânica
            )
            for peca in pecas_orçamento
            if isinstance(peca, PeçaMecânica)
        ):
            continue

    if tipo_peça_lataria is not None:

```

```
if any(
    not peca_lataria_atende_filtros(
        peca,
        categoria,
        tipo_peça_lataria
    )
    for peca in pecas_orçamento
    if isinstance(peca, PeçaLataria)
):
    continue
```

```
orçamentos_selecionados.append(orçamento)
```

```
return orçamentos_selecionados
```

```
def peca_mecânica_atende_filtros(peca, categoria, tipo_peça_mecânica):
```

```
    if categoria is not None and peca.categoria != categoria:
```

```
        return False
```

```
    if tipo_peça_mecânica is not None and peca.tipo != tipo_peça_mecânica:
```

```
        return False
```

```
    return True
```

```
def peca_lataria_atende_filtros(peca, categoria, tipo_peça_lataria):
```

```
    if categoria is not None and peca.categoria != categoria:
```

```
        return False
```

```
    if tipo_peça_lataria is not None and peca.tipo != tipo_peça_lataria:
```

```
        return False
```

```
    return True
```

```
class Orcamento:
```

```
    def __init__(self, sinistro, seguradora, data):
```

```
        self.sinistro = sinistro
```

```
        self.seguradora = seguradora
```

```
        self.data = data
```

```

def __str__(self):
    formato = '{ } {:<3} { } {:<19} { } {:<10} { }'
    mostra_formato = formato.format('|', self.sinistro.numero, '|', self.seguradora.nome, '|',
str(self.data), '|')
    return mostra_formato

def str_valores_pecas(self):
    valores_pecas_str = ""
    for indice, peca in enumerate(self.sinistro.pecas.values()):
        if indice > 0:
            valores_pecas_str += ' - '
            valores_pecas_str += f'R$ {peca.preco:.2f}'
    return valores_pecas_str

def str_atributos_pecas(self):
    atributos_pecas_str = ""
    for indice, peca in enumerate(self.sinistro.pecas.values()):
        if indice > 0:
            atributos_pecas_str += ' -- '
            if isinstance(peca, PeçaMecânica):
                atributos_pecas_str += f'{peca.categoria} - {peca.tipo} - {peca.dias_garantia} dias'
            elif isinstance(peca, PeçaLataria):
                atributos_pecas_str += f'{peca.categoria} - {peca.tipo} - {peca.cor}'
    return atributos_pecas_str

def str_atributos_pecas_filtro(self):
    atributos_pecas_str = ""
    for indice, peca in enumerate(self.sinistro.pecas.values()):
        if indice > 0:
            atributos_pecas_str += ' -- '
            if isinstance(peca, PeçaMecânica):
                atributos_pecas_str += f'{peca.categoria} - {peca.tipo}'
            elif isinstance(peca, PeçaLataria):
                atributos_pecas_str += f'{peca.categoria} - {peca.tipo}'
    return atributos_pecas_str

```



```
def str_filtro(self):
    formato = '{:>2} {} {:<11} {} {:<23} {} {:<55} {}'
    filtro_formatado = formato.format(
        str(self.seguradora.cobertura_percentual) + '%',
        '|', self.sinistro.telefone,
        '|', self.str_valores_pecas(),
        '|', self.str_atributos_pecas_filtro(),
        '|'
    )
    return self.__str__() + filtro_formatado
```

\$\$\$ src.entidades.sinistro

```
sinistros = {}
```

```
def get_sinistros ():
```

```
    return sinistros
```

```
def inserir_sinistro(sinistro):
```

```
    numero_sisnitro = sinistro.numero
```

```
    if numero_sisnitro not in sinistros.keys():
```

```
        sinistros[numero_sisnitro] = sinistro
```

```
    return True
```

```
    else:
```

```
        print('Sinistro ' + numero_sisnitro + ' já tem cadastro')
```

```
    return False
```

```
def set_sinistros(sinistros1):
```

```
    global sinistros
```

```
    sinistros = sinistros1
```

```
class Sinistro:
```

```
    def __init__(self, numero, cliente, telefone):
```

```
        self.numero = numero
```

```
        self.cliente = cliente
```

```
        self.telefone = telefone
```

```
        self.pecas = {}
```

```
    def __str__(self):
```

```
        formato = '{ } {:<3} { } {:<15} { } {:<14} { }'
```

```
        sinistro_formatado = formato.format('|', self.numero, '|', self.cliente, '|', self.telefone, '|')
```

```
        return sinistro_formatado
```

```
    def inserir_peca(self, peca):
```

```
        nome_peca = peca.nome
```

```
        if nome_peca not in self.pecas.keys(): self.pecas[nome_peca] = peca
```

```
        else: print('Nome ' + nome_peca + ' já foi cadastrada em Sinistro')
```

\$\$\$ src.interfaces.interface_textual

```
from util.gerais import imprimir_objetos, imprimir_objeto, imprimir_objetos_internos,
imprimir_objetos_associacao_filtros
from util.data import converte_str_para_data
from entidades.sinistro import Sinistro, inserir_sinistro, get_sinistros
from entidades.seguradora import Seguradora, inserir_seguradora, get_seguradoras
from entidades.pecas import PeçaMecânica, PeçaLataria
from entidades.orcamento import Orcamento, filtrar_orçamentos, get_orçamentos, criar_orçamento,
inserir_orçamento

def loop_opções_execução():
    sair_loop = False
    cabeçalho_seguradora = '\nSeguradoras : nome - cidade - cobertura percentual'
    cabeçalho_sinistro = '\nSinistro : numero - clientes - telefone'
    cabeçalho_peças_sinistros = ('\nSinistro : numero - clientes - telefone'
    + '\n - Peças : código - nome - categoria - preço - peça mecânica:[tipo - prazo de garantia] | peça
lataria:[tipo - cor]'
    + ' - mão_obra_própria')
    cabeçalho_orçamento = ('\nOrçamento : Numero do sinistro - tipo de peça - nome da seguradora'
    + ' - data do orçamento')

    while not sair_loop:
        print()
        operação = ler_str('Opções [C: Cadastrar / I: Imprimir / S: Selecionar / T: imprimir Todos /
<ENTER>: Parar]', retornar=True)
        if operação == None:
            break
        elif operação == 'C':
            opção_conteúdo = ler_str('A: Seguradoras / B: Sinistros / C: Orçamentos / <ENTER>:
retornar]',
                                     retornar=True)
            if opção_conteúdo == None:
                pass
            elif opção_conteúdo == 'A':
                loop_leitura_seguradoras()
            elif opção_conteúdo == 'B':
                loop_leitura_sinistros()
```

```

elif opção_conteúdo == 'C':
    loop_leitura_orçamentos()

elif operação == 'I':
    opção_conteúdo = ler_str('A: Seguradoras / B: Sinistros / C: Orçamentos / P: Peças do
sinistro /'
                                + '<ENTER>: retornar]',
                                retornar=True)
    if opção_conteúdo == None:
        pass
    elif opção_conteúdo == 'A':
        imprimir_objetos(cabeçalho_seguradora, get_seguradoras().values())
    elif opção_conteúdo == 'B':
        imprimir_objetos(cabeçalho_sinistro, get_sinistros().values())
    elif opção_conteúdo == 'C':
        imprimir_objetos(cabeçalho_orçamento, get_orçamentos())
    elif opção_conteúdo == 'P':
        imprimir_peças_sinistro(cabeçalho_peças_sinistros)

elif operação == 'S':
    loop_seleção_orçamentos()

elif operação == 'T':
    imprimir_objetos(cabeçalho_seguradora, get_seguradoras().values())
    imprimir_objetos(cabeçalho_sinistro, get_sinistros().values())
    imprimir_objetos(cabeçalho_orçamento, get_orçamentos())
    imprimir_peças_sinistro(cabeçalho_peças_sinistros)

def imprimir_peças_sinistro(cabeçalho_peças_sinistros):
    print(cabeçalho_peças_sinistros)
    for indice, sinistro in enumerate(get_sinistros().values()):
        imprimir_objeto(indice=indice, objeto_str=str(sinistro))
        imprimir_objetos_internos(sinistro.pecas.values())

def loop_leitura_seguradoras():
    sair_loop = False
    print('--- Leitura de Dados das Seguradoras ---')

```

```
while not sair_loop:
    seguradora = ler_seguradora()
    if seguradora is not None:
        inserir_seguradora(seguradora)
    else:
        print(' - ERRO : na leitura da seguradora')
    sair_loop = ler_sair_loop('cadastro de seguradoras')
```

```
def loop_leitura_sinistros():
    sair_loop = False
    print('--- Leitura de Dados dos Sinistros ---')
    while not sair_loop:
        sinistro = ler_sinistro()
        if sinistro is not None:
            inserir_sinistro(sinistro)
            loop_leitura_peças_sinistro(sinistro)
        else:
            print(' - ERRO : na leitura do sinistro')
        sair_loop = ler_sair_loop('cadastro de sinistro')
```

```
def loop_leitura_peças_sinistro(sinistro):
    sair_loop = False
    print('--- Leitura de Dados das Peças do Sinistro : ' + sinistro.numero + ' ---')
    while not sair_loop:
        peça = ler_peça()
        if peça is not None: sinistro.inserir_peca(peça)
        else:
            print(' - ERRO : na leitura de peça')
        sair_loop = ler_sair_loop('cadastro de peça do sinistro')
```

```
def loop_leitura_orçamentos():
    sair_loop = False
    print('--- Leitura de Dados de Orçamentos ---')
    while not sair_loop:
        orçamento = ler_orçamento()
        if orçamento is not None:
            inserir_orçamento(orçamento)
```

```
else:
    print(' - ERRO : na leitura do orçamento')
    sair_loop = ler_sair_loop('cadastro do orçamento')
```

```
def ler_sair_loop(loop):
    try:
        sair = input('\n-- sair do loop de ' + loop + ' [S]: ')
        if sair == 'S':
            return True
    except IOError:
        pass
    return False
```

```
def loop_seleção_orçamentos():
    sair_loop = False
    print('--- Seleção de Orçamentos ---')
    while not sair_loop:
        filtros, orçamentos_selecionados = selecionar_orçamentos()
        if filtros is not None:
            cabeçalho = ('Orçamento : número do sinistros - nome da seguradora - data do orçamento'
                          + '\n -- cobertura percentual - cidade'
                          + '\n - peças[categoria - tipo_peça_mecânica | categoria - tipo_peça_lataria]')
            imprimir_objetos_associacao_filtros(cabeçalho, orçamentos_selecionados, filtros)
        sair_loop = ler_sair_loop('seleção de lançamentos')
```

```
def ler_seguradora():
    nome = ler_str('nome da seguradora')
    if nome == None:
        return None
    cidade = ler_str('cidade da seguradora')
    if cidade == None:
        return None
    cobertura_percentual = ler_int_positivo('cobertura percentual da seguradora')
    if cobertura_percentual == None:
        return None
    return Seguradora(nome, cidade, cobertura_percentual)
```

```
def ler_sinistro():
    número = ler_str('numero do sinistro')
    if número == None:
        return None
    cliente = ler_str('cliente do Sinistro')
    if cliente == None:
        return None
    telefone = ler_str('telefone no sinistro [DDD + NUM]')
    if telefone == None:
        return None
    return Sinistro(número, cliente, telefone)
```

```
def ler_peça():
    código = ler_int_positivo('código da peça')
    if código == None:
        return None
    nome = ler_str('nome da peça')
    if nome == None:
        return None
    categoria = ler_categoria_peça('categoria da peça')
    if categoria == None:
        return None
    preço = ler_int_positivo('preço da peça')
    if preço == None:
        return None
    mão_obra_própria = ler_bool('mão de obra própria')
    if mão_obra_própria == None:
        return None
    espécie_peça = ler_str('espécie de peça [Pm=Peça mecânica / Pl=Peça lataria]')

    if espécie_peça == 'Pm':
        tipo = ler_tipo_peça_mecanica()
        if tipo == None:
            return None
        dias_garantia = ler_int_positivo('Dias de garantia')
        if dias_garantia == None:
            return None
```

```
return PeçaMecânica(código, nome, categoria, preço, tipo, dias_garantia, mão_obra_própria)
```

```
if espécie_peça == 'PI':
```

```
    tipo = ler_tipo_peça_lataria()
```

```
    if tipo == None:
```

```
        return None
```

```
    cor = ler_str('cor da peça')
```

```
    if cor == None:
```

```
        return None
```

```
    return PeçaLataria(código, nome, categoria, preço, tipo, cor, mão_obra_própria)
```

```
else:
```

```
    return None
```

```
def ler_orçamento():
```

```
    numero_sinistro = ler_str('numero do sinistro')
```

```
    if numero_sinistro == None:
```

```
        return None
```

```
    nome_seguradora = ler_str('nome da seguradora')
```

```
    if nome_seguradora == None:
```

```
        return None
```

```
    data_orçamento = ler_data('data da orçamento')
```

```
    if data_orçamento is None:
```

```
        return None
```

```
    return criar_orçamento(numero_sinistro, nome_seguradora, data_orçamento)
```

```
def selecionar_orçamentos():
```

```
    filtros = '\nFiltros -- '
```

```
    filtros_pecas = []
```

```
    data_mínima_orcamento = ler_data('data mínima do orçamento', filtro=True)
```

```
    if data_mínima_orcamento is not None: filtros += 'Data mínima do orcamento: ' +  
str(data_mínima_orcamento)
```

```
    valor_máximo_peca = ler_int_positivo('maior valor da peça', filtro=True)
```

```
    if valor_máximo_peca is not None: filtros += ' - Maior valor da peca: ' + str(valor_máximo_peca)
```

```
    cobertura_mínima_seguradora = ler_int_positivo('cobertura mínima da seguradora', filtro=True)
```



```
if cobertura_mínima_seguradora is not None: filtros += "\n - cobertura mínima da seguradora: ' + str(cobertura_mínima_seguradora)
```

```
prefixo_telefone_cliente = ler_str('prefixo do telefone do cliente', filtro=True)  
if prefixo_telefone_cliente is not None: filtros += (' - DDD telefone cliente: ' + str(prefixo_telefone_cliente))
```

```
categoria = ler_str('categoria da peça', filtro=True)  
if categoria is not None:  
    filtros_pecas.append('categoria: ' + str(categoria))
```

```
tipo_peça_mecânica = ler_str('tipo de peça mecânica', filtro=True)  
if tipo_peça_mecânica is not None:  
    filtros_pecas.append('tipo_peça_mecânica: ' + str(tipo_peça_mecânica))
```

```
tipo_peça_lataria = ler_str('tipo de lataria', filtro=True)  
if tipo_peça_lataria is not None:  
    filtros_pecas.append('tipo_peça_lataria: ' + str(tipo_peça_lataria))
```

```
if len(filtros_pecas) > 0:  
    filtros += "\n - peças[' + ' - '.join(filtros_pecas) + '']
```

```
orçamentos_selecionados = filtrar_orçamentos(data_mínima_orcamento,  
                                              valor_máximo_peça, cobertura_mínima_seguradora,  
                                              prefixo_telefone_cliente, categoria, tipo_peça_mecânica,  
                                              tipo_peça_lataria)  
return filtros, orçamentos_selecionados
```

```
def ler_categoria_peça(filtro=False):  
    try:  
        string = input(  
            '- tipo de peça [A=Original / B=Genuína / C=OEM ]: '  
        )  
        if len(string) == 0 and filtro: return None  
        if string == 'A': return 'Original'  
        if string == 'B': return 'Genuína'  
        if string == 'C': return 'OEM'  
    except IOError:
```

```
    pass
print('Erro na leitura da categoria de peça')
return None
```

```
def ler_tipo_peça_mecanica(filtro=False):
    try:
        string = input(
            '- tipo de peça [A=suspensão / B=direção / C=motor]: ')
        if len(string) == 0 and filtro:
            return None
        if string == 'A': return 'suspensão'
        if string == 'B': return 'direção'
        if string == 'C': return 'motor'
    except IOError:
        pass
    print('Erro na leitura do tipo de peça mecânica')
    return None
```

```
def ler_tipo_peça_lataria(filtro=False):
    try:
        string = input(
            '- tipo de peça [A=externo / B=interno]: ')
        if len(string) == 0 and filtro:
            return None
        if string == 'A': return 'externo'
        if string == 'B': return 'interno'
    except IOError:
        pass
    print('Erro na leitura do tipo de peça de lataria')
    return None
```

```
def ler_str(dado, filtro=False, retornar=False):
    try:
        string = input('- ' + dado + ': ')
        if len(string) == 0 and (filtro or retornar):
            return None
        if len(string) > 0:
```

```
        return string
    except IOError:
        pass
    print('Erro na leitura do dado: ' + dado)
    return None
```

```
def ler_int_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ': ')
        if len(string) == 0 and filtro:
            return None
        int_positivo = int(string)
        if int_positivo > 0:
            return int_positivo
    except ValueError:
        pass
    print('Erro na leitura/conversão do inteiro positivo: ' + dado)
    return None
```

```
def ler_float_positivo(dado, filtro=False):
    try:
        string = input('- ' + dado + ': ')
        if len(string) == 0 and filtro:
            return None
        float_positivo = float(string)
        if float_positivo > 0.0: return float_positivo
    except ValueError:
        pass
    print('Erro na leitura/conversão do flutuante positivo: ' + dado)
    return None
```

```
def ler_bool(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [S/N]: ')
        if len(string) == 0 and filtro:
            return None
        if string == 'S':
```

```
        return True
    elif string == 'N':
        return False
except ValueError:
    pass
print('Erro na leitura do booleano: ' + dado)
return None
```

```
def ler_data(dado, filtro=False):
    try:
        string = input('- ' + dado + ' [dd/mm/aaaa]: ')
        if len(string) == 0 and filtro:
            return None
        data = converte_str_para_data(string)
        if data is not None: return data
    except IOError:
        pass
    print('Erro na leitura da data: ' + dado)
    return None
```

\$\$\$ src.util.data

```
from datetime import date
```

```
from re import match
```

```
class Data:
```

```
    def __init__(self, dia, mês, ano):
```

```
        self.dia = dia
```

```
        self.mês = mês
```

```
        self.ano = ano
```

```
    def __str__(self):
```

```
        if self.dia < 10: data_str = '0' + str(self.dia)
```

```
        else: data_str = str(self.dia)
```

```
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
```

```
        else: data_str += "/" + str(self.mês) + "/"
```

```
        data_str += str(self.ano)
```

```
        return data_str
```

```
    def __eq__(self, data):
```

```
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return True
```

```
        return False
```

```
    def __ne__(self, data): return not self == data
```

```
    def __gt__(self, data):
```

```
        if self.ano > data.ano: return True
```

```
        elif self.ano < data.ano: return False
```

```
        if self.mês > data.mês: return True
```

```
        elif self.mês < data.mês: return False
```

```
        if self.dia > data.dia: return True
```

```
        elif self.dia < data.dia: return False
```

```
        return False
```

```
    def __lt__(self, data):
```

```
        if self.ano < data.ano: return True
```

```
        elif self.ano > data.ano: return False
```

```
if self.mês < data.mês: return True
elif self.mês > data.mês: return False
if self.dia < data.dia: return True
elif self.ano > data.ano: return False
return False
```

```
def __ge__(self, data):
    if self < data: return False
    else: return True
```

```
def __le__(self, data):
    if self > data: return False
    else: return True
```

```
def calcular_idade(self, data_referência=None):
    if data_referência is None:
        dia_atual_str, mês_atual_str, ano_atual_str = date.today().strftime("%d/%m/%Y").split('/')
        dia_referência, mês_referência, ano_referência = int(dia_atual_str), int(mês_atual_str),
int(ano_atual_str)
    else:
        dia_referência, mês_referência, ano_referência = data_referência.dia, data_referência.mês,
data_referência.ano
        idade = ano_referência - self.ano
        if mês_referência < self.mês or (mês_referência == self.mês and dia_referência < self.dia):
idade -= 1
    return idade
```

```
def converte_str_para_data(data_str):
    if not match(r'(0?[1-9]|[12][0-9]|3[01])/(0?[1-9]|1[012])/d{4}', data_str): return None
    dia, mês, ano = data_str.split('/')
    return Data(int(dia), int(mês), int(ano))
```

\$\$\$ src.util.gerais

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for indice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(indice, str(objeto))
```

```
def imprimir_objeto(índice, objeto_str):
```

```
    formato = '{} {} {}'
```

```
    ordem = índice + 1
```

```
    separador = ' _ '
```

```
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)
```

```
    print(string_formatado)
```

```
def imprimir_objetos_associação_filtros(cabeçalho, objetos, filtros=None):
```

```
    if filtros is not None: print(filtros)
```

```
    print(cabeçalho)
```

```
    for índice, objeto in enumerate(objetos):
```

```
        imprimir_objeto(índice, objeto.str_filtro())
```

```
def imprimir_objetos_internos (objetos):
```

```
    for objeto in objetos: print(' - ' + str(objeto))
```

\$\$\$ src.util.persistência_arquivo

```
import pickle
```

```
def salvar_arquivo(nome_arquivo, objetos):
```

```
    arquivo = open('../dados/' + nome_arquivo + '.bin', 'wb')
```

```
    pickle.dump(objetos, arquivo)
```

```
    arquivo.close()
```

```
def carregar_arquivo(nome_arquivo):
```

```
    try:
```

```
        arquivo = open('../dados/' + nome_arquivo + '.bin', 'rb')
```

```
        objetos = pickle.load(arquivo)
```

```
    except IOError:
```

```
        objetos = None
```

```
    return objetos
```